

Functies

INFO

Lesinhoud

In deze les leer je wat functies zijn, waarom je ze gebruikt en hoe je ze kan definiëren en aanroepen.

Doelstellingen

De student(e) kan:

- een logisch probleem opdelen in kleine onderdelen om tot een oplossing te komen.
- een webpagina dynamisch maken met behulp van Javascript (JS) en het Document Object Model (DOM).
- de basisprincipes van Javascript-programmatie (variabelen en constanten, controlestructuren, functies, arrays, objecten, DOM en DOM manipulatie, events en event handling) toepassen binnen een project.

▼ Inhoud

- [Functies](#)
 - [Waarom functies gebruiken?](#)
 - [Een functie declareren](#)
 - [Parameters en Argumenten](#)
 - [Een waarde teruggeven met return](#)
- [Andere manieren om een functie te declareren](#)
 - [De functie-expressie](#)

- [hoisting](#)
- [De arrow function](#)
- [Arrow functions in de praktijk](#)

Functies

Een functie is een herbruikbaar blok code dat een specifieke taak uitvoert.

In plaats van dezelfde code steeds opnieuw te schrijven, kan je die code in een functie plaatsen en de functie aanroepen wanneer je de taak wilt uitvoeren. Dit maakt je code overzichtelijker, korter en makkelijker te onderhouden.

Probeer identieke stukken code in je scripts zoveel mogelijk te vermijden.

Functie/Methode

Een functie is een losstaand blok code dat je zelf definieert.

```
function mijnFunctie() {  
  // code hier  
}  
mijnFunctie(); // functie aanroepen
```

js

Een methode is een functie die deel uitmaakt van een object.

```
const mijnObject = {  
  mijnMethode: function () {  
    // code hier  
  },  
};  
mijnObject.mijnMethode(); // methode (functie van het object)  
aanroepen
```

js

Waarom functies gebruiken?

- **Herbruikbaarheid:** Schrijf de code één keer en gebruik het zo vaak je wilt.
- **Organisatie:** Verdeel een complex probleem in kleinere, behapbare deeltaken.
- **Abstractie:** Je hoeft niet te weten hoe een functie precies werkt, alleen wat ze doet. Denk aan `console.log()`: je weet dat het iets toont in de console, maar de interne werking is voor jou verborgen.

Een functie declareren

Je kan een functie declareren met het `function`-keyword, gevolgd door een naam, haakjes `()`, en de code tussen accolades `{ }` (het codeblok).

```
function begroetGebruiker() {  
  // Hier komt alle code die uitgevoerd zal worden als je  
  de begroetGebruiker functie aanroept.  
  console.log("Hallo en welkom!");  
}  
  
begroetGebruiker(); // Hier wordt de functie een eerste  
keer aangeroepen  
begroetGebruiker(); // Hier wordt de functie een tweede  
keer aangeroepen
```

js

Parameters en Argumenten

Vaak wil je een functie flexibeler maken door data mee te geven. Dit doe je met `parameters`: variabelen die je definieert in de functiedeclaratie.

Wanneer je de functie aanroept, geef je waarden mee voor die `parameters`. Die waarden noemen we `argumenten`.

Als voorbeeld de functie uit het vorig voorbeeld heeft nu een `parameter` genaamd `naam`. We roepen deze functie twee keer aan, één keer met als `argument` `"Alice"` en één keer met als `argument` `"Bob"`.

```
function persoonlijkeGroet(naam) {  
  // 'naam' is hier een parameter.  
  console.log("Hallo, " + naam);  
}  
  
// "Alice" en "Bob" zijn de argumenten die we meegeven.  
persoonlijkeGroet("Alice"); // Geeft "Hallo, Alice!"  
weer.  
persoonlijkeGroet("Bob"); // Geeft "Hallo, Bob!" weer.
```

js

Een waarde teruggeven met return

Een functie kan ook een resultaat of waarde teruggeven. Hiervoor gebruik je het `return`-keyword. Zodra een `return`-statement wordt uitgevoerd, stopt de functie en wordt de waarde teruggestuurd naar de plek waar de functie werd aangeroepen.

```
<div id="output"></div>
```

html

```
// Deze functie berekent de som en geeft het resultaat  
terug.  
function berekenSom(getal1, getal2) {  
  const som = getal1 + getal2;  
  return som; // Einde van de functie, geef wat achter  
  `return` staat als resultaat terug.
```

js

```
// 'unreachable code', deze code wordt nooit uitgevoerd  
aangezien de functie stopt bij 'return'.  
console.log("Deze boodschap zie je nooit.");  
}  
  
const resultaat = berekenSom(5, 3); // Evalueert naar 8,  
8 wordt aan de variabele `resultaat` toegekend.  
const paragraaf = document.createElement("p");  
paragraaf.innerText = resultaat;  
document.body.appendChild(paragraaf);  
  
// Direct het resultaat gebruiken is ook mogelijk.  
document.querySelector("#output").innerText =  
  `De som van 10 en 20 is ${berekenSom(10, 20)}.`;
```

De som van 10 en 20 is 30.

8

Andere manieren om een functie te declareren

De functie-expressie

Naast het declareren van een functie met `function mijnFunctie(){}` , kun je in JavaScript een functie ook toewijzen aan een variabele. Dit wordt een `functie-expressie` genoemd. In dit geval maak je vaak een `anonieme functie` (een functie zonder naam) en sla je die op in een variabele. De variabele wordt dan gezien als de functienaam, waarmee je de functie kan aanroepen.

js

```
// Dit is een functie-expressie.  
// De functie zelf heeft geen naam, maar wordt opgeslagen  
in de variabele 'groet'.  
// Deze functie kan pas aangeroepen worden na de regel  
waar hij geïnitieerd wordt!  
const groet = function () {  
  console.log("Hallo vanuit een functie-expressie!");  
};  
  
// Je roept de functie aan via de naam van de variabele.  
groet();
```

hoisting

De term **hoisting** betekent "zet de code bovenaan van de code al klaar".

Functie-declaratie (wel gehoist)

js

```
zegHallo(); // Werkt  
  
function zegHallo() {  
  console.log("Hallo!");  
}
```

Functie-expressie (niet gehoist)

js

```
zegHallo(); // Werkt niet!  
  
const zegHallo = function () {  
  console.log("Hallo!");  
};
```

Arrow function (niet gehoist)

```
zegHallo(); // Werkt niet!  
  
const zegHallo = () => {  
  console.log("Hallo!");  
};
```

Wat altijd werkt, in alle drie de situaties, is om eerst de functie aan te maken en daarna de functie pas aan te roepen.

De arrow function

Naast de traditionele `function-declaraties` en `-expressies`, introduceerde ES6 (een modernere versie van JavaScript) een kortere en compactere manier om functies te schrijven: de `arrow function`. Arrow functions zijn vooral handig voor korte, `anonieme functies`, zoals die vaak gebruikt worden in array-methoden zoals `forEach()`, `find()` en `filter()`, etc.

Syntax

Laten we een traditionele functie-expressie vergelijken met een arrow function.

```
// Traditionele functie-expressie  
const kwadraat = function (x) {  
  return x * x;  
};
```

```
// Arrow function equivalent  
const kwadraat = (x) => {
```

```
    return x * x;  
};
```

De syntax wordt nog korter in de volgende gevallen:

Impliciete return: Als de functie alleen een `return`-statement bevat, mag je de accolades `{}` en het `return`-keyword weglaten.

```
// bovenstaande functie nog korter geschreven  
const kwadraat = (x) => x * x;
```

js

Ook de ronde haakjes zijn optioneel als er maar één parameter is.

Eén parameter: Als de functie precies één parameter heeft, zijn de ronde haakjes `()` rond de parameter optioneel.

```
const persoonlijkeGroet = (naam) => `Hallo, ${naam}!`;  
// is hetzelfde als: (geen haakjes nodig met maar één  
// param)  
// const persoonlijkeGroet = naam => `Hallo, ${naam}!`;
```

js

Geen parameters: Als de functie geen parameters heeft, moet je lege haakjes `()` gebruiken.

```
const geefWillekeurigGetal = () => Math.random() * 10;
```

js

Meerdere parameters: Als de functie meerdere parameters heeft, moeten de haakjes `()` gebruikt worden.

```
const som = (a, b) => a + b;
```

js

Arrow functions in de praktijk

Hieronder een voorbeeld met een named functie als callback:

Merk op dat het de naam van de functie is die als argument wordt doorgegeven aan `forEach`, zonder deze aan te roepen met haakjes `()`.

```
<div id="prijs-container"></div>
```

html

```
const prijzen = [10, 20, 30, 40];

function toonPrijs(prijs) {
  const prijsParagraaf = document.createElement("p");
  prijsParagraaf.innerText = prijs;
  const prijsContainer = document.querySelector("#prijs-
container");
  prijsContainer.appendChild(prijsParagraaf);
}

prijzen.forEach(toonPrijs);
```

js

10

20

30

40

Hieronder een voorbeeld met een gewone anonieme functie als callback:

```
<div id="prijs-container"></div>
```

html

```
const prijzen = [10, 20, 30, 40];

prijzen.forEach(function (prijs) {
  const prijsParagraaf = document.createElement("p");
  prijsParagraaf.innerText = prijs;
  const prijsContainer = document.querySelector("#prijs-
container");
  prijsContainer.appendChild(prijsParagraaf);
});
```

js

10

20

30

40

Hieronder hetzelfde voorbeeld, maar dan met een anonieme arrow function als callback:

```
<div id="prijs-container"></div>
```

html

```
const prijzen = [10, 20, 30, 40];

prijzen.forEach((prijs) => {
  const prijsParagraaf = document.createElement("p");
  prijsParagraaf.innerText = prijs;
  const prijsContainer = document.querySelector("#prijs-
container");
  prijsContainer.appendChild(prijsParagraaf);
});
```

js

```
});
```

10

20

30

40

Anonieme arrow functions krijgen de voorkeur wanneer je een korte functie nodig hebt die je maar één keer gebruikt.

forEach versus for

Dit is de eerste keer dat we `forEach()` gebruiken in plaats van de `for` - notaties uit het hoofdstuk waar je kennismaat met iteratieve statements. Met het statement `prijzen.forEach(toonPrijs)` lopen we over alle prijzen, en voor elke prijs roepen we de functie `toonPrijs()` aan met die prijs als enige argument. Anders gezegd: dit is hetzelfde als:

```
for (const prijs of prijzen) {  
  toonPrijs(prijs);  
}
```

js

Previous page
[Oefeningen](#)

Next page
[Oefeningen](#)